



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

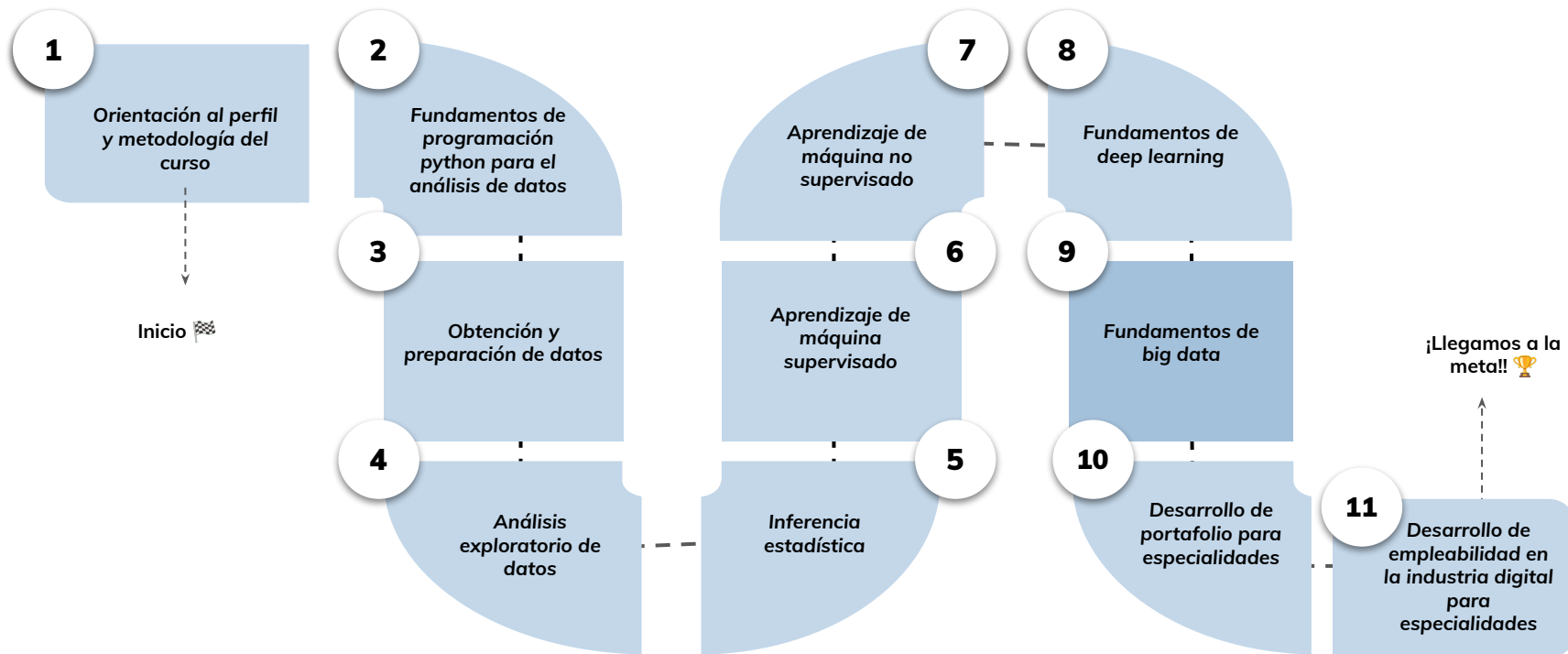


› Procesamiento de datos: estructurados- Parte I

Aprendizaje Esperado 4: Manipular grandes volúmenes de datos utilizando spark sql para el procesamiento de grandes volúmenes de datos estructurados.

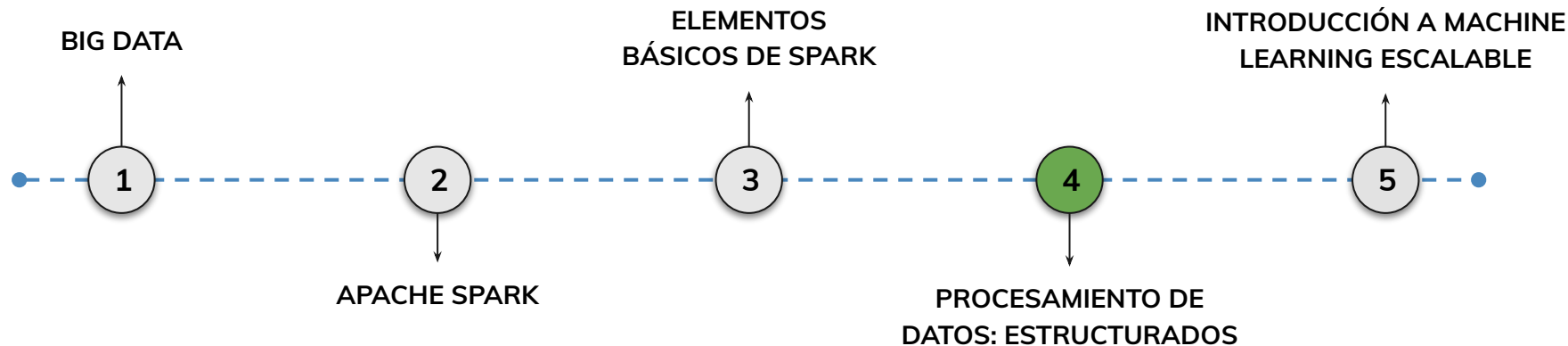
Hoja de ruta

¿Cuáles skills conforman el programa? Fundamentos de Ciencia de Datos



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

4.

Procesamiento de datos estructurados con Spark SQL

Aprenderemos a utilizar Spark SQL y DataFrames para consultar, transformar y optimizar grandes volúmenes de datos estructurados de manera distribuida.

Introducción y fundamentos de Spark SQL

Qué es Spark SQL y sus beneficios.

Diferencias con el trabajo directo sobre RDD.

DataFrames y consultas SQL en Spark

Creación de DataFrames desde distintas fuentes.

Operaciones de filtrado, ordenamiento y agregación.



Objetivos de aprendizaje

¿Qué aprenderás?



- Comprender el funcionamiento y ventajas de Spark SQL para datos estructurados.
- Crear y manipular DataFrames desde diversas fuentes de datos.
- Ejecutar consultas SQL en grandes volúmenes de datos distribuidos.
- Integrar datos de distintos formatos (CSV, JSON, Parquet) y sistemas (HDFS, S3, JDBC).
- Utilizar el motor Catalyst y el proyecto Tungsten para optimizar consultas.

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Optimizamos operaciones con RDD usando `reduceByKey` y evitando `groupByKey`.
- Aplicamos variables broadcast y acumuladores para mejorar rendimiento.
- Leímos y escribimos datos en formatos CSV, JSON y Parquet.
- Conectamos RDD con Spark SQL y revisamos módulos adicionales.

Procesamiento de datos: estructurados

Procesamiento de Datos Estructurados con Apache Spark

En los proyectos de análisis de datos, gran parte de la información que se procesa se encuentra en forma estructurada, es decir, organizada en tablas, con filas y columnas que representan registros y atributos. Este tipo de datos es habitual en bases de datos relacionales, hojas de cálculo, archivos CSV, JSON y Parquet, entre otros. Gestionar, consultar y transformar estos datos de forma eficiente es un desafío esencial en entornos de Big Data.

Introducción a Apache Spark

Apache Spark no solo ofrece capacidades avanzadas para el procesamiento distribuido de datos no estructurados, sino que también proporciona un potente módulo llamado **Spark SQL**, diseñado específicamente para trabajar con datos estructurados y semiestructurados.

Spark SQL amplía las capacidades de Spark, permitiendo la consulta de datos mediante sentencias SQL, la manipulación de **DataFrames** y la integración con diversas fuentes de datos.



Propósito de este manual

Comprensión de Spark SQL

Entender el funcionamiento de Spark SQL y sus componentes principales para el procesamiento de datos estructurados.

Ventajas de DataFrames

Conocer las ventajas de utilizar DataFrames para el manejo eficiente de datos estructurados.

Técnicas de optimización

Explorar las técnicas de optimización que permiten acelerar el procesamiento de consultas en entornos distribuidos.

Interoperabilidad y formatos

Abordar la interoperabilidad entre RDD y DataFrames, así como el uso de formatos de almacenamiento eficientes como JSON y Parquet.

Importancia del procesamiento de datos estructurados

El procesamiento de datos estructurados con Spark representa un recurso clave para analistas de datos, ingenieros y científicos de datos que necesitan manejar grandes volúmenes de información estructurada, integrando consultas SQL tradicionales con la potencia del procesamiento distribuido.

¿Qué es Spark SQL?

Spark SQL es un módulo de **Apache Spark** diseñado para trabajar con datos estructurados y semiestructurados. Ofrece una interfaz que permite realizar consultas utilizando el lenguaje SQL estándar, así como operaciones programáticas sobre estructuras de datos distribuidas llamadas **DataFrames** y **Datasets**. Este módulo combina la familiaridad del lenguaje SQL con la potencia del procesamiento distribuido de Spark.

Evolución de Spark SQL

Antes de Spark SQL

Spark se centraba en el uso de **RDDs (Resilient Distributed Datasets)**, que ofrecían un alto nivel de flexibilidad pero requerían escribir un código más complejo para realizar consultas estructuradas.

Actualidad

Con Spark SQL, se simplificó el acceso y manipulación de datos estructurados, permitiendo ejecutar consultas SQL tradicionales sobre grandes conjuntos de datos distribuidos.

1

2

3

Introducción de Spark SQL

La incorporación de Spark SQL en el ecosistema Spark respondió a la necesidad de procesar datos estructurados de manera más eficiente y sencilla.

Beneficios principales de Spark SQL



Consultas SQL sobre datos distribuidos

Permite ejecutar consultas SQL tradicionales sobre grandes conjuntos de datos distribuidos, sin necesidad de mover los datos a un sistema gestor de bases de datos relacional (RDBMS).



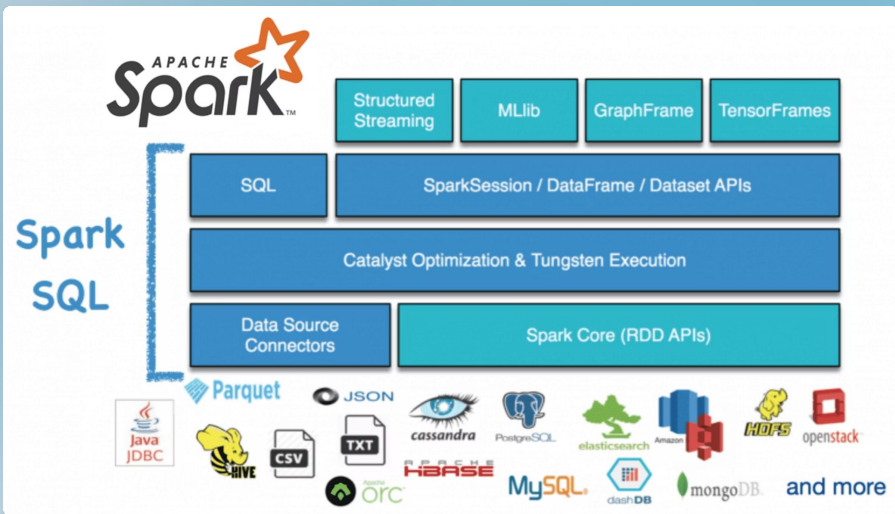
API unificada

Proporciona una **API unificada** que permite a los usuarios trabajar con datos utilizando SQL, DataFrames y Datasets de forma intercambiable.



Soporte para múltiples fuentes de datos

Soporta múltiples fuentes de datos, incluyendo archivos **CSV**, **JSON**, **Parquet**, **ORC**, bases de datos JDBC y sistemas de almacenamiento distribuido como **HDFS**, **S3** y **Azure Blob Storage**.



Optimización en Spark SQL

Catalyst Optimizer

Utiliza un motor de consulta llamado **Catalyst Optimizer**, que analiza y mejora los planes de ejecución de las consultas para maximizar la eficiencia.

Proyecto Tungsten

Spark SQL aprovecha el proyecto **Tungsten** para optimizar el uso de la memoria y la CPU durante la ejecución.

Aplicaciones prácticas de Spark SQL



Limpieza de datos

Facilita la limpieza y preparación de grandes volúmenes de datos estructurados para su posterior análisis.



Generación de reportes

Facilita la creación de informes y visualizaciones a partir de grandes conjuntos de datos.



Integración de fuentes

Permite combinar datos provenientes de diferentes fuentes en un único análisis integrado.



Análisis exploratorio

Permite realizar análisis exploratorio de datos a gran escala y con altos niveles de rendimiento.

Características de Spark SQL

Spark SQL se ha consolidado como uno de los módulos más utilizados de Apache Spark, gracias a sus características que combinan flexibilidad, rendimiento y facilidad de uso. Estas características permiten a los desarrolladores y analistas de datos trabajar con grandes volúmenes de datos estructurados de manera eficiente.

Soporte para múltiples lenguajes



Python

Permite escribir código en Python utilizando la API de PySpark, ideal para científicos de datos y analistas.



Scala

Lenguaje nativo de Spark, ofrece el mejor rendimiento y acceso completo a todas las funcionalidades.



Java

Soporte completo para desarrollo en Java, ampliamente utilizado en entornos empresariales.



R

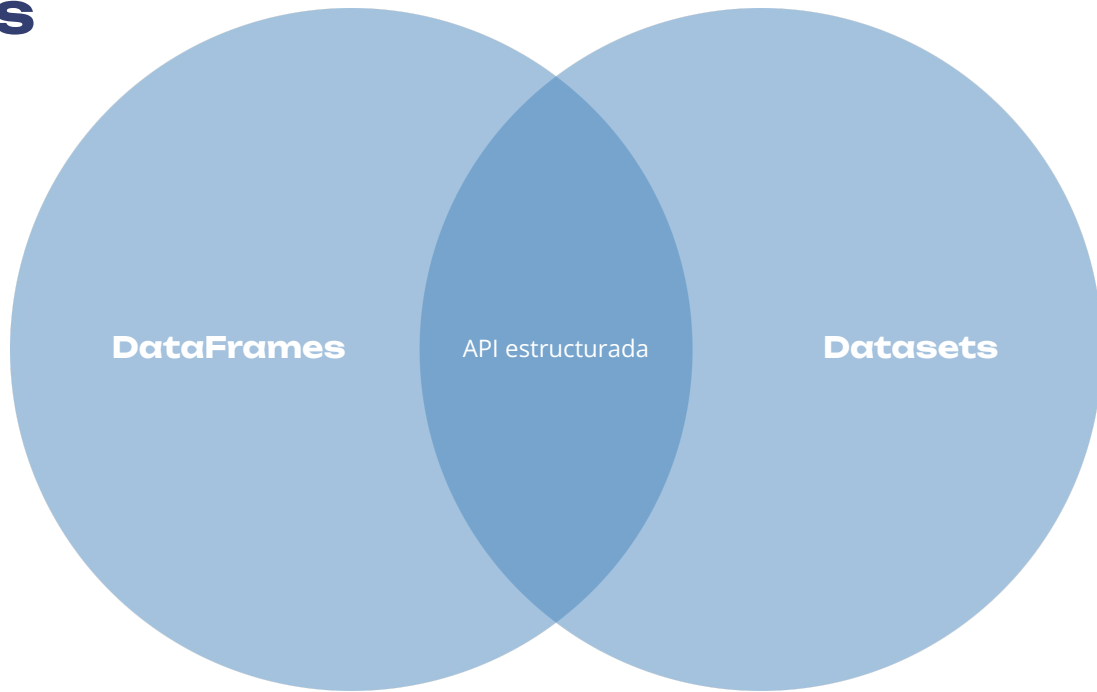
Integración con R para análisis estadísticos y visualizaciones avanzadas.

Esta diversidad de lenguajes facilita la adopción de Spark SQL en equipos multidisciplinarios con conocimientos en diferentes tecnologías.

Una característica importante es la posibilidad de ejecutar **consultas SQL tradicionales** sobre grandes volúmenes de datos. Spark SQL permite realizar consultas complejas utilizando la sintaxis estándar de SQL, lo que resulta familiar para analistas y profesionales acostumbrados a trabajar con bases de datos relacionales.



DataFrames y Datasets



Spark SQL también destaca por su capacidad para trabajar con **DataFrames y Datasets**, estructuras de datos distribuidas que permiten manipular información estructurada de manera eficiente. Los DataFrames representan tablas con columnas y filas, similares a una hoja de cálculo o a una tabla en una base de datos.

Integración con fuentes de datos



CSV

Archivos de valores separados por comas, ampliamente utilizados para intercambio de datos tabulares.



JSON

Formato ligero de intercambio de datos, ideal para información semiestructurada.



JDBC

Conexión con bases de datos relacionales como MySQL, PostgreSQL y Oracle.



Almacenamiento distribuido

Integración con HDFS, Amazon S3 y Azure Blob Storage.

Motor de optimización Catalyst

Una característica clave de Spark SQL es su **motor de optimización Catalyst**. Este motor analiza las consultas y genera planes de ejecución optimizados que minimizan el movimiento de datos y aprovechan al máximo los recursos del clúster. Esto se traduce en una mejora significativa del rendimiento de las consultas.

Proyecto Tungsten



Gestión eficiente de memoria

Optimiza el uso de memoria mediante técnicas avanzadas de gestión y distribución.



Generación de código a bajo nivel

Genera código optimizado específico para cada consulta, mejorando el rendimiento.



Optimización de CPU

Aprovecha al máximo los recursos de procesamiento disponibles en el clúster.

Gracias a estas optimizaciones, Spark SQL es capaz de superar a muchos sistemas tradicionales de procesamiento de datos.

Consultas interactivas

Otra ventaja de Spark SQL es la posibilidad de realizar **consultas interactivas**. Los usuarios pueden conectarse a un clúster Spark y ejecutar consultas SQL en tiempo real, obteniendo resultados de manera rápida y eficiente.

¿Qué es una sesión Spark?

En el contexto de **Apache Spark**, la **sesión Spark** (o **SparkSession**) es el punto de entrada principal para interactuar con las funcionalidades de Spark, especialmente con las relacionadas al módulo **Spark SQL** y al manejo de datos estructurados mediante **DataFrames** y **Datasets**.

Evolución de la sesión Spark

Antes de Spark 2.0

Las aplicaciones Spark utilizaban un objeto llamado **SparkContext** para inicializar el entorno de ejecución.

1

2

Spark 2.0

Introducción de la clase `SparkSession` que unificó y simplificó la creación y configuración de las aplicaciones.

Actualidad

`SparkSession` integra tanto el `SparkContext` como el `SQLContext` en un solo objeto, facilitando el desarrollo de aplicaciones.

3

Creación de una sesión Spark

La creación de una sesión Spark es sencilla y requiere especificar algunos parámetros básicos, como el nombre de la aplicación y el modo de ejecución (local o en clúster).

Por ejemplo, en **PySpark**, se puede crear una sesión Spark de la siguiente manera:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MiAplicacionSQL") \
    .master("local[*]") \
    .getOrCreate()
```

Acceso a componentes de la sesión Spark

SparkContext

Una vez creada la sesión, es posible acceder al **SparkContext** a través de sus atributos.

```
spark_context = spark.sparkContext
```

SQLContext

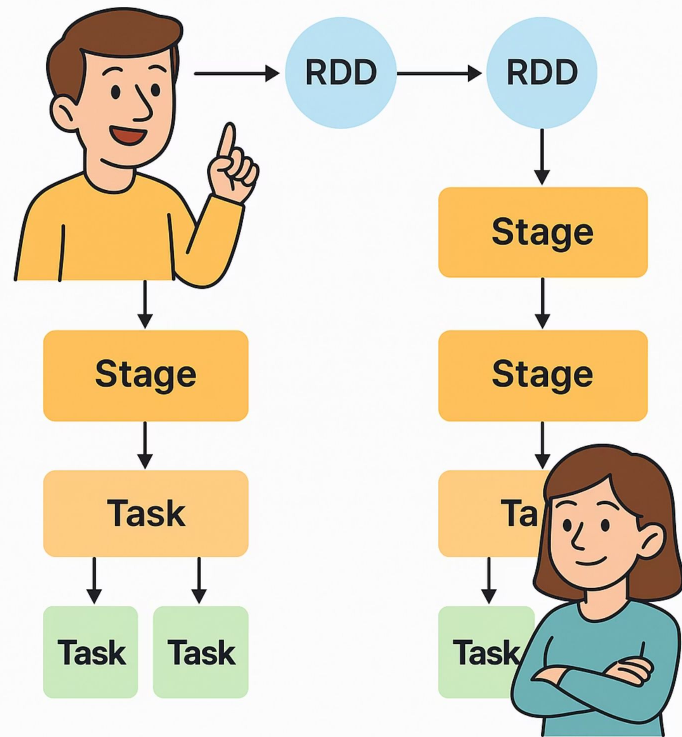
También se puede acceder al **SQLContext** para operaciones específicas de SQL.

```
sql_context = spark.sqlContext
```

Configuración de la sesión Spark

La sesión Spark también permite configurar parámetros importantes, como la cantidad de memoria asignada a los ejecutores, la cantidad de núcleos utilizados y la ubicación de los archivos de entrada y salida.

Basic of Spark Directed Acyclic Graph



Division of stages into tasks enables parallel processing

Estado de la aplicación

Otra ventaja de la sesión Spark es que mantiene el estado de la aplicación durante toda su ejecución. Esto significa que los usuarios pueden ejecutar múltiples consultas y transformaciones sobre los datos sin necesidad de reiniciar el contexto de ejecución.



Cierre de la sesión Spark

Es importante cerrar la sesión Spark al finalizar la aplicación para liberar los recursos asignados. Esto se realiza mediante el método `stop()`:

```
spark.stop()
```

¿Qué es un DataFrame Spark?

Uno de los elementos clave en el procesamiento de datos estructurados con **Apache Spark** es el concepto de **DataFrame**. Un **DataFrame** es una colección distribuida de datos organizados en columnas, similar a una tabla en una base de datos relacional o a un DataFrame en Python (Pandas) o R.



Características de los DataFrames

Abstracción de alto nivel

Ofrece una abstracción de alto nivel que permite manipular y consultar datos estructurados de manera eficiente, utilizando tanto funciones de programación como consultas SQL.

Información estructural

Añade información estructural sobre las columnas y sus tipos de datos, lo que permite realizar optimizaciones adicionales mediante el motor **Catalyst**.

Soporte para diversos tipos

Puede contener datos de cualquier tipo, como números, cadenas de texto, fechas y estructuras más complejas como arrays y mapas.

Funcionamiento de los DataFrames

El funcionamiento de un DataFrame se basa en el principio de **evaluación perezosa** (lazy evaluation). Esto significa que las operaciones sobre un DataFrame no se ejecutan inmediatamente, sino que se registran como un plan lógico de ejecución. Este plan se optimiza y ejecuta únicamente cuando se solicita una acción que requiera un resultado, como `show()` o `count()`.

Ventajas de los DataFrames

Compatibilidad con múltiples lenguajes

Compatibilidad con múltiples lenguajes de programación, incluyendo **Python (PySpark)**, **Scala**, **Java** y **R**.

Operaciones avanzadas

Permiten realizar operaciones avanzadas sobre los datos, como filtrado, agregación, ordenamiento, unión y combinación de conjuntos de datos.

Interoperabilidad

Son interoperables con otros componentes de Spark, como Spark SQL, MLlib (biblioteca de aprendizaje automático) y GraphX (biblioteca de grafos).



Live Coding

¿En qué consistirá la Demo?

Exploraremos cómo crear una sesión Spark, generar DataFrames desde archivos y ejecutar consultas SQL distribuidas.

1. Crear una SparkSession en PySpark.
2. Cargar datos desde CSV y JSON en DataFrames.
3. Registrar un DataFrame como vista temporal para usar SQL.
4. Ejecutar una consulta SQL con agregaciones y filtros.
5. Comparar el rendimiento con y sin optimizaciones Catalyst/Tungsten.



Momento:

Time-out!



5 -10 min.





Ejercicio

Consultas distribuidas con Spark SQL

Consultas distribuidas con Spark SQL

Contexto: 🙌

Aplicarás Spark SQL para realizar consultas complejas sobre grandes volúmenes de datos estructurados.

Consigna: 📝

1. Crea una sesión Spark y carga un conjunto de datos (CSV o Parquet).
2. Convierte los datos en un DataFrame y regístralo como vista temporal.
3. Escribe y ejecuta tres consultas SQL que incluyan filtros, agregaciones y ordenamientos.
4. Explica cómo Catalyst optimiza la ejecución de estas consultas.

Paso a paso: ⚙️

1. Configura la sesión Spark.
2. Carga los datos y crea el DataFrame.
3. Define las vistas temporales necesarias.
4. Ejecuta y analiza los resultados de las consultas.

Tiempo 🕒: 45 Minutos

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ **Comprendimos qué es Spark SQL y su importancia en Big Data.**
- ✓ **Aprendimos a crear y manipular DataFrames para datos estructurados.**
- ✓ **Ejecutamos consultas SQL distribuidas sobre grandes volúmenes de datos.**
- ✓ **Conocimos las optimizaciones internas Catalyst y Tungsten.**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

